

# Práctica 1: Criptografía Clásica

Javier López Cano      Mario García Pascual

October 28, 2020

## 1 Introducción

En esta práctica hemos implementado distintos criptosistemas para cifrar y descifrar ficheros siguiendo distintos métodos de cifrado. Entre estos métodos están el método afín (afin.c), una variante mejorada de este cifrado afín ligeramente más segura (afin\_var.c), cifrado de Vignere (vignere.c), un cifrador sencillo de flujo (stream.c) y un criptosistema con un cifrado de permutación doble similar al cifrado de Hill (permutación.c).

Para la obtención de los parámetros de entrada de todos estos criptosistemas hemos creado unas funciones de parseado de argumentos muy similares en todos los criptosistemas. Estas leen los parámetros de entrada y los guardan en las variables correspondientes, tomando en cuenta si estos parámetros son obligatorios u opcionales, y estableciendo los valores por defecto en caso de que sean opcionales y no se hayan proporcionado (por ejemplo, stdin y stdout como ficheros de entrada y salida en caso de no haberse especificado ficheros concretos). Estas funciones de parseado de argumentos se llaman "parse\_args" y "load\_args" en todos los criptosistemas, un ejemplo de estas funciones son las empleadas en afin.c:

```
1 int parse_args(int argc, char *argv[]){
2     int i;
3     i = 1;
4     while (i < argc){
5         if (argv[i][0] != '-'){
6             printf("Error: sintaxis\n");
7             return ERR;
8         }
9         if (strcmp(argv[i], "-C") == 0){
10             args[MODE_] = i;
11         }else if (strcmp(argv[i], "-D") == 0){
12             args[MODE_] = i;
13         }else if (strcmp(argv[i], "-m") == 0){
14             args[M_] = ++i;
15         }else if (strcmp(argv[i], "-a") == 0){
16             args[A_] = ++i;
17         }else if (strcmp(argv[i], "-b") == 0){
18             args[B_] = ++i;
19         }else if (strcmp(argv[i], "-i") == 0){
20             args[IN_] = ++i;
21         }else if (strcmp(argv[i], "-o") == 0){
22             args[OUT_] = ++i;
23         }else{
24             printf("Error: argumento no identificado\n");
```

```

25         return ERR;
26     }
27
28     if (argc == i){
29         printf("Error: sintaxis\n");
30         return ERR;
31     }else{
32         i++;
33     }
34 }
35 for (i = 0; i < NARGS; i++){
36     if (args[i] == OB){
37         printf("Error: falta argumento obligatorio\n");
38         return ERR;
39     }
40 }
41 return OK;
42 }
43
44 int load_args(char *argv[]){
45     /* Argumentos obligatorios */
46     if (strcmp(argv[args[MODE_]], "-C") == 0){
47         mode = ENC;
48     }else // if (strcmp(argv[i], "-D") == 0){
49         mode = DEC;
50     }
51     mpz_init(m);
52     mpz_init(a);
53     mpz_init(b);
54     mpz_set_str(m, argv[args[M_]], 10);
55     mpz_set_str(a, argv[args[A_]], 10);
56     mpz_set_str(b, argv[args[B_]], 10);
57     /* Argumentos opcionales */
58     if (args[IN_] == OP){
59         in = stdin;
60     }else{
61         in = fopen(argv[args[IN_]], "r");
62     }
63     if (args[OUT_] == OP){
64         out = stdout;
65     }else{
66         out = fopen(argv[args[OUT_]], "w");
67     }
68     return OK;
69 }

```

## 2 Cifrado Afín

El criptosistema que emplea cifrado afín se encuentra en el fichero "afin.c". Para desarrollarlo, en primer lugar hemos creado una función que comprueba si los argumentos "a", "b", y "m" son válidos para este cifrado. Para ser válidos deben cumplir que "a" sea mayor o igual a 1 y que "b" sea mayor o igual a 0 y menor que "m" ya que de otro modo existiría otro "b" entre 0 y "m" que daría lugar a un cifrado equivalente. También comprobamos que "a" y "b" son coprimos, para lo cual empleamos el algoritmo de euclides implementado en el archivo euclides.c. Tras esto comenzamos a llevar a cabo el cifrado en sí, que consiste en, recorrer el fichero de entrada caracter a caracter y a cada uno de ellos aplicarle la fórmula:

$$c = a * c + b \text{ mod } m$$

Debemos tener en cuenta que estamos tratando con caracteres y no con números enteros, y, por tanto, para poder realizar el cálculo de forma correcta, debemos restarle al caracter leído el caracter 'A' para obtener números entre 0 y "m", y, tras aplicar la fórmula, volver a sumar este caracter para conseguir así el valor ASCII del caracter codificado. Durante todo este proceso de cifrado, así como durante el descifrado que explicaremos posteriormente, hemos empleado la librería GMP del lenguaje C, pues trabajamos con números mayores a los que entran en una simple variable "int", de este modo el código para cifrar un caracter es:

```

1  mpz_t res1, res2;
2  c = c - 'A';
3  if(c < 0 | mpz_cmp_ui(m, c) <= 0){
4      printf("Caracter en texto plano no pertenece al alfabeto.\n");
5      mpz_clear(a);
6      mpz_clear(b);
7      mpz_clear(m);
8      return ERR;
9  }
10 mpz_init(res1);
11 mpz_init(res2);
12 mpz_mul_ui(res1, a, c);
13 mpz_add(res2, res1, b);
14 mpz_mod(res1, res2, m);
15 c = 'A' + mpz_get_ui(res1);
16 mpz_clear(res1);
17 mpz_clear(res2);
18 return c;

```

Esto lo repetimos para cada caracter del fichero de entrada, e imprimimos el resultado en el fichero de salida, como se ve en la función "affine\_encode" del archivo afin.c.

Para el proceso de descifrado comprobamos también que los argumentos son válidos para el cifrado afín y realizamos un proceso análogo pero aplicando la fórmula inversa:

$$c = a^{-1} * (c - b) \bmod m$$

Donde  $a^{-1}$  es el inverso multiplicativo de "a" en módulo "m".

Por tanto el código para descifrar un caracter es:

```

1  mpz_t res1, res2, res3;
2  c = c - 'A';
3  if(c < 0 | mpz_cmp_ui(m, c) <= 0){
4      printf("Caracter en texto plano no pertenece al alfabeto.\n");
5      mpz_clear(a);
6      mpz_clear(b);
7      mpz_clear(m);
8      return ERR;
9  }
10 mpz_init(res1);
11 mpz_init(res2);
12 mpz_init(res3);
13 mpz_invert(res1, a, m);
14 mpz_ui_sub(res2, c, b);
15 mpz_mul(res3, res1, res2);

```

```

16  mpz_mod(res1, res3, m);
17  c = 'A' + mpz_get_ui(res1);
18  mpz_clear(res1);
19  mpz_clear(res2);
20  mpz_clear(res3);
21  return c;

```

Y, del mismo modo que en el caso del cifrado, repetimos el proceso para cada caracter del texto cifrado e imprimimos el resultado en el fichero de salida mediante la función "affine\_decode".

Por último, en este fichero `afin.c` incluimos también el programa principal "main" que llama a las distintas funciones de parseado de argumentos y al encriptador o desencriptador según sea necesario, e imprime los errores pertinentes en caso de que se den.

## 2.1 Uso del programa

Hemos seguido el formato especificado en el enunciado, que es el siguiente:

```
afin {-C|-D} {-m m} {-a a} {-b b} [-i file_in] [-o file_out]
```

donde los parámetros son:

- el modo: `-C` o `-D`, para cifrar o descifrar.
- el tamaño del alfabeto: `-m` seguido de un número que indica el tamaño del alfabeto que en nuestro caso será siempre 26 para incluir las letras mayúsculas de la 'A' a la 'Z'.
- el valor numérico "a" del cifrado afín (clave multiplicativa): `-a` seguido de un número entero mayor o igual a 1.
- el valor numérico "b" del cifrado afín (clave aditiva): `-b` seguido de un número entero entre 0 (incluido) y "m" (sin incluir).
- el fichero de entrada: `-i` seguido del nombre del fichero. Si no se introduce se usa la entrada estándar
- el fichero de salida: `-o` seguido del nombre del fichero. Si no se introduce se usa la salida estándar

## 3 Variación más segura del Cifrado Afín

En el fichero "afin\_var.c" hemos implementado una variación del criptosistema afín más segura en cuanto al número de claves. Para aumentar este número de claves, hemos requerido al usuario que los argumentos "a" y "b" sean tuplas de números, en vez de un único número cada uno. Cada uno de los números de la tupla debe seguir cumpliendo los mismos requisitos que en el cifrado afín, y, en este caso cada uno de los 2 números de la tupla "a" debe ser coprimo con ambos

número de la tupla "b".

La variación aplicada consiste en ir variando el par de datos (a, b) con el que se cifra o descifra cada caracter, de este modo, cada caracter de un grupo de 4 caracteres consecutivos se cifrará con un par de claves distinto. En el caso de cigrado, las fórmulas para cifrar cada cacter de un grupo de 4 será:

Primer caracter:  $c = a_1 * c + b_1$   
 Segundo caracter:  $c = a_2 * c + b_1$   
 Tercer caracter:  $c = a_1 * c + b_2$   
 Cuarto caracter:  $c = a_2 * c + b_2$

En el caso del descifrado, las fórmulas serán:

Primer caracter:  $c = a_1^{-1} * (c - b_1)$   
 Segundo caracter:  $c = a_2^{-1} * (c - b_1)$   
 Tercer caracter:  $c = a_1^{-1} * (c - b_2)$   
 Cuarto caracter:  $c = a_2^{-1} * (c - b_2)$

Donde  $a_1^{-1}$  y  $a_2^{-1}$  son el inverso multiplicativo de  $a_1$  y  $a_2$  en módulo "m" respectivamente.

Para lograr cifrar y descifrar empleando este método, simplemente a las funciones "encode\_char" y "decode\_char", que cifran y descifran un caracter, le pasamos, además del caracter original, un contador "i" que va del 0 al 3, y seleccionamos el par de claves (a, b) en función de dicho contador. Como ejemplo proporcionamos la función "encode\_char" del fichero "afin\_var.c":

```

1  int encode_char(int c, int i){
2  mpz_t res1, res2;
3  c = c - 'A';
4  if(c < 0 | mpz_cmp_ui(m, c) <= 0){
5      printf("Caracter en texto plano no pertenece al alfabeto.\n");
6      mpz_clear(a1);
7      mpz_clear(b1);
8      mpz_clear(a2);
9      mpz_clear(b2);
10     mpz_clear(m);
11     return ERR;
12 }
13 mpz_init(res1);
14 mpz_init(res2);
15 if(i < 2){
16     mpz_mul_ui(res1, a1, c);
17 }else{
18     mpz_mul_ui(res1, a2, c);
19 }
20 if((i % 2) == 0){
21     mpz_add(res2, res1, b1);
22 }else{
23     mpz_add(res2, res1, b2);
24 }
25 mpz_mod(res1, res2, m);

```

```

26 c = 'A' + mpz_get_ui(res1);
27 mpz_clear(res1);
28 mpz_clear(res2);
29 return c;
30 }

```

La función "decode\_char" es análoga con las fórmulas del descifrado.

Las funciones "affine\_var\_encode" y "affine\_var\_decode" son las que recorren el fichero de entrada caracter a caracter incrementando el contador cada vez, y manteniéndolo en módulo 4 para obtener siempre números entre 0 y 3.

Este nuevo método de cifrado basado en afín es más robusto en cuanto al número de claves, lo que se puede comprobar estimando el número de claves posibles del cifrado afín y de esta variación.

La clave del cifrado afín consta de 2 números "a" y "b", por tanto el número de claves que pueden existir será el número de posibles "a" por el número de posibles "b". El número "a" puede tomar el valor de cualquier número entero positivo coprimo con "b", este número se trata de un valor numerable que llamaremos "n", por su parte "b" toma valores enteros entre 0 (incluido) y "m" (sin incluir), como estamos trabajando con un alfabeto conformado por las letras mayúsculas de la 'A' a la 'Z',  $m = 26$ , por tanto:

$$claves_{afin} = 26 * n$$

En esta nueva variante, la clave está formada también por "a" y "b" pero cada uno de estos ahora es una tupla, es decir, un conjunto de 2 números en los que importa el orden. Por tanto "a" está compuesta por  $a_1$  y  $a_2$  cada uno de los cuales puede tomar n valores (donde n al igual que antes es el número de enteros positivos coprimos con "b"), por tanto la tupla "a" puede tomar  $n^2$  valores. Del mismo modo, "b" es una tupla formada por  $b_1$  y  $b_2$  donde cada uno de estos toma valores entre 0 (incluido) y 26 (sin incluir) lo cual son 26 valores, por tanto:

$$claves_{afin\_var} = 26^2 * n^2 = 676 * n^2$$

Esto supone una cantidad enormemente mayor de claves, pues hemos elevado al cuadrado la cantidad de claves del cifrado afín.

Sin embargo este cifrado sigue muy lejos de ser perfecto y, por tanto, sigue siendo criptoanalizable fácilmente mediante métodos estadísticos como el análisis de frecuencias. Para realizar este análisis debemos presuponer el idioma del texto, sabiendo que tiene 26 caracteres. Para este criptoanálisis vamos a realizar un ejemplo con el español.

Tomamos un texto cifrado suficientemente grande para poder realizar el criptoanálisis de forma correcta, nosotros hemos decidido tomar el quijote (modificado quitando espacios, signos de puntuación y cambiando todo a mayúsculas) encriptado con nuestro "afin\_var.c" proporcionado en el fichero "quijote\_enc\_afin\_var". Sabemos que este criptosistema encripta con un par de claves diferente cada caracter de cada grupo de cuatro, por tanto nos vamos a ir fijando en los caracteres

según cual sea su posición módulo 4.

En castellano la letra más repetida es la letra "E", con esto en mente empezamos a contar las letras que más aparecen en los caracteres del texto cifrado cuya posición módulo 4 es cero. Descubrimos así que la letra más repetida es la "O", que es el caracter 14. A partir de esta información deducimos que:

$$a_1 * c + b_1 \bmod 26 = 14$$

En los caracteres cuya posición módulo 4 es 1 el más repetido es la "F"=5, entonces:

$$a_1 * c + b_2 \bmod 26 = 5$$

En los caracteres cuya posición módulo 4 es 2 el más repetido es la "M"=12, entonces:

$$a_2 * c + b_1 \bmod 26 = 12$$

En los caracteres cuya posición módulo 4 es 3 el más repetido es la "D"=3, entonces:

$$a_2 * c + b_2 \bmod 26 = 3$$

A partir de estas 4 ecuaciones formamos un sistema, que nos da como resultado:

$$a_1 = 17; a_2 = 23; b_1 = 24; b_2 = 15$$

Por tanto la clave empleada para cifrar el texto (y necesaria para descifrarlo) es:

$$a = (17, 23); b = (24, 15)$$

Y, efectivamente si intentamos descifrarlo, obtenemos el texto del quijote.

### 3.1 Uso del programa

Hemos seguido un formato análogo al especificado en el enunciado, éste es:

```
afin_var {-C|-D} {-m m} {-a a1 a2} {-b b1 b2} [-i file_in] [-o file_out]
```

donde los parámetros son:

- el modo: `-C` o `-D`, para cifrar o descifrar.
- el tamaño del alfabeto: `-m` seguido de un número que indica el tamaño del alfabeto que en nuestro caso será siempre 26 para incluir las letras mayúsculas de la 'A' a la 'Z'.
- la tupla "a" que contiene 2 claves multiplicativas para el cifrado afín: `-a` seguido de 2 números enteros mayores o iguales a 1.
- la tupla "b" que contiene 2 claves aditivas para el cifrado afín: `-b` seguido de 2 números enteros entre 0 (incluido) y "m" (sin incluir).
- el fichero de entrada: `-i` seguido del nombre del fichero. Si no se introduce se usa la entrada estándar
- el fichero de salida: `-o` seguido del nombre del fichero. Si no se introduce se usa la salida estándar

## 4 Cifrado de Vigenere

Nuestra implementación del criptosistema que emplea el cifrado de Vigenere se encuentra en el fichero "vigenere.c". Para la implementación de este criptosistema no hemos empleado la librería GMP, pues trabajamos siempre con números relativamente pequeños. Aquí, del mismo modo que en los casos anteriores, implementamos las funciones de parseado de argumentos, así como las funciones "vigenere.encode" y "vigenere.decode" que recorren el fichero de entrada carácter a carácter cifrando o descifrando cada uno con las funciones "encode\_char" y "decode\_char" siguiendo este tipo de cifrado, e imprime el resultado en el fichero de salida.

Para lograr el cifrado de vigenere, en la función "vigenere.encode" creamos un contador que aumenta con cada carácter que leemos del fichero de entrada, y pasamos este contador, junto con el carácter leído a la función "encode\_char", que, con estos parámetros así como con la longitud del alfabeto y la longitud de la clave, cifra el carácter según la fórmula:

$$c = (c + k_i) \bmod m$$

Donde  $k_i$  es el carácter en posición  $i$ -ésima de la clave ("i" es el contador), y "m" es el tamaño del alfabeto.

Teniendo en cuenta que debemos restar 'A' tanto al carácter leído como a  $k_i$  para trabajar con números entre 0 y  $m - 1$ , y se debe sumar luego de nuevo tras la operación para obtener el código ASCII del carácter resultado, la función "encode\_char" queda:

```
1  int encode_char(int c, int i){
2      return (c + key[i % key_len] - 2*'A') % ALPH_SIZE + 'A';
3  }
```

El descifrado se realiza de forma análoga con la función inversa, que es:

$$c = (c - k_i) \bmod m$$

En este fichero encontramos también la función "is\_valid\_str" que comprueba que la clave es válida para realizar el cifrado, es decir, que solo contiene caracteres pertenecientes al alfabeto; y el programa principal (main) que llama a las distintas funciones de parseado de argumentos, comprueba que la clave sea válida mediante la función explicada anteriormente, y llama a las funciones de cifrado o descifrado según sea necesario.

### 4.1 Uso del programa

Hemos seguido el formato especificado en el enunciado, que es el siguiente:

```
vigenere {-C|-D} {-m m} {-k clave} [-i file_in] [-o file_out]
```

donde los parámetros son:

- el modo: -C o -D, para cifrar o descifrar.

- el tamaño del alfabeto: `-m` seguido de un número que indica el tamaño del alfabeto que en nuestro caso será siempre 26 para incluir las letras mayúsculas de la 'A' a la 'Z'.
- la clave de cifrado: `-k` seguido de una cadena de caracteres pertenecientes al alfabeto que contenga la clave de cifrado.
- el fichero de entrada: `-i` seguido del nombre del fichero. Si no se introduce se usa la entrada estándar
- el fichero de salida: `-o` seguido del nombre del fichero. Si no se introduce se usa la salida estándar

## 5 Criptoanálisis del Vigenere

Vamos a explicar de forma sucinta cómo se implementan los métodos del Índice de Coincidencia (IC) y el test de Kasiski. Además, se explica cómo obtener la clave conociendo la longitud de la misma y el idioma original del texto.

### 5.1 Índice de Coincidencia para hallar la longitud de la clave

En primer lugar: en el contexto del cifrado de Vigenere, llamamos **subtexto** al subconjunto del texto que es cifrado por el mismo caracter de la clave. Supongamos que queremos comprobar si la clave es de tamaño  $k$ . Una clave de tamaño  $k$  nos genera  $k$  subtextos que han sido cifrados con el mismo caracter. Para cada subtexto, calculamos  $p_i$  como la proporción de caracteres de  $i$  que aparecen en el subtexto. Una vez tenemos todos los  $p_i$ , el índice de coincidencia se obtiene

$$IC = \sum p_i^2.$$

Muchas veces se calcula  $p_i = f_i/t$ , donde  $f_i$  es la frecuencia del caracter y  $t$  la longitud total del subtexto. Después podemos tomar la media del IC de cada subtexto. La longitud de clave que obtenga el mayor IC será la candidata al longitud de clave Vigenere. Además, comparando el valor del IC máximo con el de el español o el inglés podemos tener candidato a idioma del cifrado.

#### 5.1.1 Uso del programa

Hemos seguido el formato especificado en el enunciado, que es el siguiente:

```
ic {-l tamaño_a_probar} [-i file_in] [-o file_out]
```

donde los parámetros son:

- el tamaño de clave a probar: `-l` seguido de un número entero positivo.
- el fichero de entrada: `-i` seguido del nombre del fichero. Si no se introduce se usa la entrada estándar

- el fichero de salida: `-o` seguido del nombre del fichero. Si no se introduce se usa la salida estándar

Vamos a ver algunas salidas del programa. El fichero `qcif.txt` contiene el texto del Quijote cifrado con la clave `QUETAL`, de 6 caracteres.

```
1 ./ic -l 5 -i qcif.txt
2 Tamano clave a probar: 5
3 qcif.txt
4 stdout
5 ----- Resultados
6 Subtexto 0      IC = 0.043765
7 Subtexto 1      IC = 0.043708
8 Subtexto 2      IC = 0.043681
9 Subtexto 3      IC = 0.043847
10 Subtexto 4     IC = 0.043760
11 ----- IC medio = 0.043752
12 Diferencia con IC ingles: 0.021248
13 Diferencia con IC espanol: 0.034248
14 Diferencia con IC aleatorio: 0.021248
15 ----- Conclusiones
```

Aquí hemos ejecutado el programa para probar con un tamaño de clave 5. Lo primero que aparece son los argumentos con los que hemos llamado al programa, que aparece en todas nuestras implementaciones. Lo siguiente es una sección de resultados, donde podemos ver el IC que se ha obtenido para cada subtexto. Al estar probando con un tamaño de clave igual a 5 se nos han generado 5 subtextos. A continuación aparece el IC medio. Luego, la diferencia del IC medio con el IC del inglés, el español y el idioma aleatorio. Por último, aparece la sección Conclusiones, que en este caso está vacía al no haber sacado ninguna conclusión.

```
1 ./ic -l 6 -i qcif.txt
2 Tamano clave a probar: 6
3 qcif.txt
4 stdout
5 ----- Resultados
6 Subtexto 0      IC = 0.075491
7 Subtexto 1      IC = 0.075002
8 Subtexto 2      IC = 0.075186
9 Subtexto 3      IC = 0.075352
10 Subtexto 4     IC = 0.075280
11 Subtexto 5     IC = 0.075123
12 ----- IC medio = 0.075239
13 Diferencia con IC ingles: 0.010239
14 Diferencia con IC espanol: 0.002761
15 Diferencia con IC aleatorio: 0.010239
16 ----- Conclusiones
17 Ojo! Dif. con IC espanol < 0.005000
```

En este caso hemos probado con 6, el tamaño de la clave real. Podemos ver como el IC de cada subtexto es mucho más alto, así como el IC medio. La diferencia con los índices verdaderos de los idiomas es mucho menor. Por último, en conclusiones se nos advierte de que la distancia con el IC del español es especialmente baja: menor que un threshold establecido a 0.005. El IC medio alto nos indica que el tamaño de la clave podría ser el correcto, mientras que la diferencia pequeña con el IC del español nos dice que el texto cifrado podría estar en español originalmente.

Mostramos ahora otro ejemplo. El contenido del fichero `qcif9.txt` es el Quijote cifrado con la clave `CROQUETAS`, de longitud 9. Se puede observar lo mismo que el apartado anterior.

```

1 ./ic -l 9 -i qcif9.txt
2 Tamano clave a probar: 9
3 qcif9.txt
4 stdout
5 ----- Resultados
6 Subtexto 0      IC = 0.074981
7 Subtexto 1      IC = 0.075332
8 Subtexto 2      IC = 0.075067
9 Subtexto 3      IC = 0.075816
10 Subtexto 4     IC = 0.074989
11 Subtexto 5     IC = 0.075065
12 Subtexto 6     IC = 0.075473
13 Subtexto 7     IC = 0.075120
14 Subtexto 8     IC = 0.075349
15 ----- IC medio = 0.075243
16 Diferencia con IC ingles: 0.010243
17 Diferencia con IC espanol: 0.002757
18 Diferencia con IC aleatorio: 0.010243
19 ----- Conclusiones
20 Ojo! Dif. con IC espanol < 0.005000

```

### 5.1.2 Qué pasa cuando el tamaño de la clave se acerca al tamaño del texto

El texto del Quijote que estoy utilizando en esta sección tiene 802022 caracteres. Para tamaño de clave 800000 obtenemos un IC medio de 0.998788; para 400000, se obtiene 0.517869; para 100000, se obtiene 0.158517. Cuando tenemos longitudes de clave tan grandes, los subtextos tienen muy pocos caracteres, hasta llegar al caso límite en el que cada subtexto tiene tan solo un caracter, obteniendo un IC de 1.

Esta pregunta me ha parecido muy rara y me da la impresión de que se refería a lo que pasa cuando el tamaño de clave que probamos se va acercando al tamaño real. Si se refiriese a esto realmente, lo que pasa se puede ver en la sección anterior: en el tamaño real el IC se dispara.

## 5.2 Test de Kasiski para la longitud de clave

El test de Kasiski consiste en buscar repeticiones de  $n$ -gramas en el texto cifrado. Los lenguajes tienen algunos conjuntos de palabras que se repiten mucho, ya sea como palabras sueltas o como partes de palabras. La cuestión es que si estas palabras aparecen en el original a una distancia que sea un múltiplo de la longitud de clave, entonces se verán igual en el texto cifrado, pues están de alguna forma *sincronizadas*. El test de Kasiski consiste en buscar estas repeticiones de  $n$ -gramas en el texto, mirar la distancia entre ellas, y con un poco de suerte encontrar que las distancias tienen un divisor común. Esto muchas veces no es posible, ya que siempre se puede colar un  $n$ -grama igual pero que no provenga del mismo conjunto de letras, por tanto hay que buscar algo así como un *divisor común probable*.

La idea para implementar este test es:

1. generar  $n$ -gramas aleatorios
2. buscar repeticiones en el texto cifrado

3. si hay repeticiones, almacenar las distancias
4. continuar haciendo esto hasta tener suficientes distancias
5. entonces, intentar buscar un divisor común a la mayoría de ellas
6. ese divisor común probable es el candidato a longitud de clave

### 5.2.1 Uso del programa

Hemos seguido el formato especificado en el enunciado, que es el siguiente:

```
kasiski {-l long_ngrama} [-i file_in] [-o file_out]
```

donde los parámetros son:

- el tamaño de los  $n$ -gramas a probar: `-l` seguido de un número entero positivo.
- el fichero de entrada: `-i` seguido del nombre del fichero. Si no se introduce se usa la entrada estándar
- el fichero de salida: `-o` seguido del nombre del fichero. Si no se introduce se usa la salida estándar

## 5.3 Índice de Coincidencia para hallar la clave

... conociendo la longitud de clave y el idioma del texto plano. En este caso tenemos que hacer algo similar para cada subtexto. Llamamos  $v_i$  a la proporción verdadera del carácter  $i$  en el idioma original. Dado un subtexto, para cada carácter  $j$  posible que nos sirva de desplazamiento, calculamos un índice de coincidencia asociado

$$IC_j = \sum v_i p_{i+j\%26}.$$

Entonces el  $j$  que alcance el máximo  $IC_j$  es nuestro candidato a ser el carácter que ha cifrado el subtexto. Hacemos esto para todos los subtextos y obtenemos una clave candidata.

### 5.3.1 Uso del programa

No se nos daba formato para este programa, encargado de hallar la clave de cifrado Vigenere conocido el tamaño de la clave y el idioma original, por lo que hemos usado esta:

```
ic_key {-ENG | -SPA} {-l tamano_a_probar} [-i file_in] [-o file_out]
```

donde los parámetros son:

- el idioma original: `-ENG` o `-SPA` dependiendo si queremos suponer que el idioma original es el español o el inglés.
- el tamaño de clave a probar: `-l` seguido de un número entero positivo.

- el fichero de entrada: `-i` seguido del nombre del fichero. Si no se introduce se usa la entrada estándar
- el fichero de salida: `-o` seguido del nombre del fichero. Si no se introduce se usa la salida estándar

```

$ ./criptoanalisis -i /Universidad/cripto/pis -ic_key -SPA -t 6 -t qicf.txt
Idioma hipotesis a probar: 0
Tamaño clave a probar: 6
qicf.txt
stdout:
----- Tabla char 0-ésimo de la clave
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
0.044 0.038 0.042 0.045 0.045 0.043 0.042 0.036 0.031 0.032 0.031 0.025 0.047 0.035 0.033 0.037 0.078 0.038 0.034 0.035 0.040 0.025 0.032 0.032 0.035 0.036
----- max = Q
----- Tabla char 1-ésimo de la clave
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
0.032 0.031 0.035 0.036 0.043 0.038 0.042 0.045 0.045 0.043 0.042 0.036 0.031 0.033 0.031 0.025 0.046 0.035 0.033 0.037 0.078 0.038 0.034 0.035 0.046 0.025
----- max = U
----- Tabla char 2-ésimo de la clave
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
0.047 0.035 0.033 0.037 0.078 0.038 0.034 0.035 0.046 0.025 0.032 0.033 0.035 0.036 0.044 0.038 0.042 0.045 0.045 0.043 0.042 0.036 0.031 0.032 0.031 0.025
----- max = E
----- Tabla char 3-ésimo de la clave
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
0.031 0.035 0.036 0.044 0.038 0.042 0.045 0.045 0.043 0.042 0.036 0.031 0.032 0.031 0.025 0.046 0.035 0.033 0.037 0.078 0.038 0.033 0.035 0.046 0.025 0.032
----- max = T
----- Tabla char 4-ésimo de la clave
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
0.078 0.038 0.034 0.035 0.046 0.025 0.032 0.032 0.035 0.036 0.044 0.038 0.042 0.045 0.045 0.043 0.042 0.036 0.031 0.033 0.031 0.031 0.025 0.047 0.035 0.033 0.037
----- Tabla char 5-ésimo de la clave
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
0.043 0.042 0.036 0.031 0.032 0.031 0.025 0.047 0.035 0.033 0.037 0.078 0.038 0.034 0.035 0.046 0.025 0.032 0.031 0.035 0.036 0.044 0.038 0.042 0.045 0.045 0.046
----- max = L
----- Conclusiones
La clave derivada del criptoanálisis es: QUETAL

```

Figure 1: Salida del programa para obtener la clave

De nuevo, lo primero que vemos son los argumentos con los que hemos llamado al programa. A continuación, vemos que para cada caracter de la clave que queremos obtener se imprime una tabla que tiene un valor para cada letra del alfabeto. Esos valores son los  $IC_j$  que hemos mencionado antes: el IC del subtexto suponiendo que se esta cifrando con la letra  $j$ . Vemos que al terminar cada tabla también aparece el caracter para el que se alcanza el máximo. Al finalizar todas las tablas, se imprime la clave derivada del criptoanálisis, es decir, aquella construida a partir de los máximos de cada tabla. En este caso, la clave derivada es QUETAL, de tamaño 6, y se correspondía al ejemplo que hemos mencionado antes del Quijote cifrado con esta misma clave.

Ahora vamos a ver otra salida para un texto diferente. En este caso, el fichero `lazzcif.txt` contiene el Lazarillo de Tormes cifrado con la clave HOLAMUNDO, de 9 letras.

```

$ ./criptoanalisis -i /Universidad/cripto/pis -ic_key -SPA -t 9 -t lazzcif.txt
Idioma hipotesis a probar: 0
Tamaño clave a probar: 9
lazzcif.txt
stdout:
----- Tabla char 0-ésimo de la clave
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
0.032 0.033 0.026 0.047 0.036 0.035 0.036 0.076 0.038 0.033 0.034 0.045 0.024 0.032 0.032 0.037 0.036 0.044 0.039 0.042 0.045 0.045 0.042 0.041 0.036 0.038
----- max = H
----- Tabla char 1-ésimo de la clave
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
0.042 0.045 0.044 0.042 0.040 0.035 0.029 0.032 0.033 0.026 0.048 0.037 0.034 0.036 0.077 0.038 0.033 0.034 0.045 0.024 0.032 0.032 0.037 0.036 0.045 0.039
----- max = O
----- Tabla char 2-ésimo de la clave
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
0.043 0.040 0.036 0.031 0.032 0.031 0.025 0.047 0.036 0.034 0.037 0.077 0.038 0.033 0.034 0.045 0.024 0.032 0.032 0.036 0.037 0.044 0.038 0.042 0.046 0.045
----- max = L
----- Tabla char 3-ésimo de la clave
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
0.077 0.037 0.033 0.034 0.046 0.024 0.032 0.031 0.037 0.036 0.044 0.038 0.043 0.046 0.044 0.042 0.041 0.036 0.030 0.033 0.032 0.026 0.048 0.036 0.034 0.037
----- max = A
----- Tabla char 4-ésimo de la clave
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
0.044 0.043 0.040 0.035 0.030 0.033 0.032 0.025 0.046 0.036 0.035 0.037 0.077 0.039 0.033 0.034 0.044 0.024 0.032 0.032 0.037 0.037 0.044 0.039 0.042 0.045
----- max = P
----- Tabla char 5-ésimo de la clave
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
0.032 0.032 0.037 0.036 0.044 0.039 0.042 0.045 0.045 0.043 0.040 0.036 0.030 0.032 0.031 0.025 0.047 0.037 0.034 0.037 0.078 0.038 0.033 0.034 0.045 0.024
----- max = U
----- Tabla char 6-ésimo de la clave
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
0.045 0.045 0.043 0.040 0.035 0.031 0.032 0.031 0.025 0.048 0.037 0.034 0.037 0.077 0.038 0.032 0.033 0.046 0.024 0.032 0.032 0.038 0.036 0.044 0.039 0.042
----- max = N
----- Tabla char 7-ésimo de la clave
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
0.036 0.034 0.037 0.077 0.038 0.033 0.034 0.045 0.024 0.032 0.030 0.038 0.036 0.044 0.038 0.043 0.045 0.040 0.035 0.038 0.033 0.032 0.026 0.048 0.044 0.039
----- max = O
----- Tabla char 8-ésimo de la clave
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
0.043 0.045 0.045 0.043 0.041 0.035 0.030 0.031 0.032 0.025 0.048 0.037 0.034 0.037 0.077 0.037 0.033 0.033 0.045 0.024 0.032 0.032 0.037 0.037 0.044 0.039
----- max = O
----- Conclusiones
La clave derivada del criptoanálisis es: HOLAMUNDO

```

Figure 2: Salida del programa para obtener la clave

Esta vez aparecen más tablas, ya que el tamaño de la clave es mayor y el número de subtextos será mayor. El programa deriva la clave correctamente, al igual que en el caso anterior.

Otra observación de las tablas es que, al igual que el IC se dispara cuando la longitud de clave real se alcanza, el IC del carácter correcto de la clave también se dispara.

## 6 Cifrado de Flujo

Para implementar el criptosistema de cifrado de flujo (en "stream.c"), hemos empleado el generador de números pseudoaleatorios ya implementado por defecto en C en la función "rand". Esta función genera números pseudoaleatorios entre 0 y un valor MAX\_RANDOM a partir de un valor inicial o semilla que se establece a través de la función "srand".

Para poder emplear este generador de forma correcta para nuestro criptosistema, simplemente hemos creado una función "rand\_" muy sencilla que simplemente nos proporciona el número que nos devuelve "rand" en módulo el tamaño del alfabeto, de modo que siempre tenemos números pseudoaleatorios entre 0 y 25 (ambos incluidos).

Tras esto, el cifrado, implementado en la función "stream\_encode", consiste únicamente en establecer la semilla del generador aleatorio al valor de la clave que se nos pasa por argumento, mediante la función "srand", y leer el fichero de entrada carácter a carácter, encriptando cada uno de ellos (llamando a la función "encode\_char") mediante la siguiente fórmula:

$$c = (c + r) \bmod m$$

Dónde "r" es el número pseudoaleatorio obtenido por "rand\_", y "m" es la longitud del alfabeto.

De este modo, la función "stream\_encode" queda:

```
1  int stream_encode(){
2      int c, i = 0;
3      srand(key);
4      while ((c = fgetc(in)) != EOF && IS_IN_ALPH(c))
5          fputc(encode_char(c, rand_()), out);
6  }
```

Y "encode\_char" simplemente hace la suma con módulo, teniendo en cuenta que debemos restarle antes 'A' al carácter leído, y sumárselo de nuevo tras obtener el resultado.

El descifrado, de nuevo, se realiza de forma análoga con la excepción de que se emplea la fórmula inversa:

$$c = (c - r) \bmod m$$

Finalmente, este fichero, al igual que los criptosistemas anteriores, contiene el programa principal que llama a las funciones de parseado de argumentos y a las funciones de cifrado y descifrado según sea necesario, proporcionando así la

funcionalidad requerida.

La fortaleza de este criptosistema radica en el hecho de que la clave de cifrado varía para cada caracter, al ir generandose otra con el generador de números pseudoaleatorios tras cifrar cada caracter; por este hecho, se trata de un criptosistema que, teóricamente, es difícil de atacar por un criptoanalista. Sin embargo, el hecho de que el generador de claves no es completamente aleatorio sino que genera números "pseudo" aleatorios, hace que las claves tengan un cierto periodo, por lo que se repiten tras un cierto número de caracteres; y, a esto hay que sumarle que la función "srand" solo admite argumentos de tipo "unsigned int" limitando significativamente el número de claves iniciales posibles a una cantidad en torno a  $2^{16}$  claves posibles.

Debido a esto, si el generador pseudoaleatorio genera claves con un periodo relativamente pequeño y el criptoanalista dispone de un texto suficientemente grande, podría descubrir el periodo con que se repiten las claves del mismo modo que descubriría la longitud de la clave en un cifrado de vigenere (por el método de Kasiski). Una vez supiese la longitud de la clave, podría descifrar la secuencia de claves (por ejemplo, de forma similar a lo realizado para la variante más segura del cifrado afín; o, más fácilmente aún si se conoce alguna palabra o frase que aparezca en el texto plano), y tras esto descifrar el texto. También sería posible realizarlo por fuerza bruta de forma bastante rápida, siempre que se conozca el generador pseudoaleatorio empleado, debido al escaso número de claves iniciales posibles.

Además, otro problema para la seguridad que hace que este criptosistema sea fácilmente atacable, es que, al aplicar un módulo sobre el resultado de la función "rand" los números obtenidos dejan de ser homogéneos, por lo que determinados números son más probables que otros.

## 6.1 Uso del programa

Hemos seguido el formato especificado en el enunciado, que es el siguiente:

```
stream {-C|-D} {-m m} {-k clave} [-i file_in] [-o file_out]
```

donde los parámetros son:

- el modo: `-C` o `-D`, para cifrar o descifrar.
- el tamaño del alfabeto: `-m` seguido de un número que indica el tamaño del alfabeto que en nuestro caso será siempre 26 para incluir las letras mayúsculas de la 'A' a la 'Z'.
- la semilla para la generación de las claves de cifrado: `-k` seguido de un número entero positivo.
- el fichero de entrada: `-i` seguido del nombre del fichero. Si no se introduce se usa la entrada estándar
- el fichero de salida: `-o` seguido del nombre del fichero. Si no se introduce se usa la salida estándar

## 7 Producto de criptosistemas permutación

El criptosistema de este apartado aparece bien explicado en el enunciado, pero vamos a darle una vuelta más. Las claves del criptosistema son dos permutaciones,

$$\begin{aligned}K_1 &: \{0, \dots, M-1\} \rightarrow \{0, \dots, M-1\}, \\K_2 &: \{0, \dots, N-1\} \rightarrow \{0, \dots, N-1\},\end{aligned}$$

del número de filas y el número de columnas respectivamente. Una permutación  $\pi$  que actúa sobre  $n$  elementos induce una matriz  $P_\pi = (p_{ij}) \in \mathcal{M}(n, n)$  de tal modo que  $p_{ij} = 1$  si  $\pi(i) = j$ , y  $p_{ij} = 0$  en caso contrario. (Tal y como la definimos,  $P_\pi$  es una matriz de permutación de filas.) Siguiendo este procedimiento, las claves  $K_1$  y  $K_2$  inducen dos matrices de permutación  $P_1 \in \mathcal{M}(M, M)$  y  $P_2 \in \mathcal{M}(N, N)$ , es decir, de tamaño  $M$  la de filas y  $N$  la de columnas.

Ahora veamos cómo se cifra y descifra un bloque. Los bloques son matrices de texto  $M \times N$ , luego podemos ver un bloque  $B$  como una matriz en  $\mathcal{M}(M, N)$ . Dicho esto, las funciones de cifrado y descifrado serían

$$\begin{aligned}C_{(K_1, K_2)}(B) &= P_1 B P_2^T, \\D_{(K_1, K_2)}(B) &= P_1^{-1} B (P_2^T)^{-1},\end{aligned}$$

pero dado que sabemos que las matrices permutación son ortogonales y por tanto cumplen  $P_\pi^{-1} = P_\pi^T$ , podemos reescribir la función de descifrado como

$$D_{(K_1, K_2)}(B) = P_1^T B P_2.$$

Habiendo descrito el criptosistema de esta manera, es trivial demostrar que  $D_{(K_1, K_2)} \circ C_{(K_1, K_2)} = \text{id}$ , y que no importa el orden en el que apliquemos los pasos que se explican en el enunciado, tanto en cifrado como en descifrado, puesto que la multiplicación de matrices es asociativa. Además, apunta a una posible implementación del criptosistema mediante multiplicación de matrices.

En cuanto al posible criptoanálisis al que se puede someter a este criptosistema, sobra decir que el propuesto en el apartado opcional consigue reventar el sistema sin mucha dificultad, pero bajo el supuesto de que se tiene la máquina cifradora para realizar cifrados a gusto, que es un supuesto muy fuerte. Otras opciones son las que tienen en común los cifrados por permutación. De primeras, aplicando el método de IC sobre todo el texto podríamos conocer el idioma original, ya que permutar las letras no altera la proporción de los símbolos. Una vez conocido el idioma, y sabiendo que es un criptoanálisis de permutación, se nos abren más oportunidades.

### 7.1 Uso del programa

Hemos seguido el formato especificado en el enunciado, que es el siguiente:

```
permutacion {-C|-D} {-k1 clave} {-k2 clave} [-i file_in] [-o file_out]
```

donde los parámetros son:

- el modo: `-C` o `-D`, para cifrar o descifrar
- la permutación de filas: `-k1` seguido de los números de la permutación separados por espacios. La permutación ha de ser válida y sólo deberá contener números de 0 a  $M - 1$ .
- la permutación de columnas: `-k2` seguido de los números de la permutación separados por espacios. La permutación ha de ser válida y sólo deberá contener números de 0 a  $N - 1$
- el fichero de entrada: `-i` seguido del nombre del fichero. Si no se introduce se usa la entrada estándar
- el fichero de salida: `-o` seguido del nombre del fichero. Si no se introduce se usa la salida estándar

Además, cambiando el archivo `permutacion.h` se puede modificar el número de filas y columnas del cifrador, que están almacenadas como macros `M` y `N` en el fichero mencionado.

Un ejemplo de uso cifrando la *Sonatina* de Ruben Darío con parámetros  $M = N = 4$ ,  $K_1 = (1, 0, 3, 2)$ ,  $K_2 = (3, 2, 1, 0)$ :

```
1 ./permutacion -C -k1 1 0 3 2 -k2 3 2 1 0 -i sntn.txt -o sntncif.txt
2 Modo: 1
3 1 0 3 2
4 3 2 1 0
5 sntn.txt
6 sntncif.txt
7 head -c 50 sntncif.txt
8 ECNIRPALRTATSEASTEUQETSIPALARDNELASECNIRRIPSUSOAC
9 tail -c 50 sntncif.txt
10 OCNEAETRELETREDNESOIBALSOSEBNUNOCNROMAEDOBQMBBRLW
11 ./permutacion -D -k1 1 0 3 2 -k2 3 2 1 0 -o sntncifdes.txt -i sntncif.txt
12 Modo: 2
13 1 0 3 2
14 3 2 1 0
15 sntncif.txt
16 sntncifdes.txt
17 head -c 50 sntncifdes.txt
18 LAPRINCESAESTATRISTEQUETENDRALAPRINCESALOSSUSPIROS
19 tail -c 50 sntncifdes.txt
20 UERTEAENCENDERTELOSLABIOSCONUNBESODEAMORNWLRBBMQB
```

Nótese como al mirar el final del texto descifrado se ha añadido unos pocos caracteres de ruido. Esto es porque, salvo si se es muy afortunado y la longitud del texto es un múltiplo de  $MN$ , en general hay que completar el último bloque del texto con caracteres aleatorios para proceder al cifrado/descifrado.

## 8 Criptoanálisis mediante texto claro escogido del doble cifrado de permutación

Para dar una explicación de cómo realizar el criptoanálisis propuesto en el enunciado vamos a empezar por hablar de un conocido puzle, o acertijo

matemático, que se suele exponer para demostrar lo útiles que pueden ser las representaciones *únicas y compactas*. El problema en cuestión es:

The King has 1000 wines, 1 of which is poisoned. He needs to identify the poisoned wine as soon as possible, and with the least resources, so he hires the protagonist, a Mathematician. The king offers you his expendable servants to help you test which wine is poisoned.

The poisoned wine is very potent, so much that one molecule of the wine will cause anyone who drinks it to die. However, it is slow-acting. The nature of the slow-acting poison means that there is only time to test one "drink" per servant. (A drink may be a mixture of any number of wines) (Assume that the King needs to know within an hour, and that any poison in the drink takes an hour to show any symptoms)

What is the minimum amount of servants you would need to identify the poisoned wine?

La solución que se suele dar es que son necesarios tantos sirvientes como la longitud del número de vinos en base 2, a saber, el techo de  $k = \log_2(1000) = 10$ . Etiquetamos los vinos del 0 al 999, tomamos el vino  $v$ , y lo expresamos en base 2, obteniendo una ristra única de  $k$  0s y 1s. Ahora usamos esta ristra del siguiente modo: si el bit  $j$ -ésimo está a 1, el sirviente  $j$  bebe el vino  $v$ ; si está a 0, no lo bebe. De esta manera, **identificamos cada vino de forma única con los sirvientes que lo beben**. Luego, una vez pasada la hora que tarda el vino en mostrar síntomas, conocemos los sirvientes que han muerto y recuperamos el vino en común que han bebido todos ellos escribiendo el número correspondiente a activar los bits que corresponden a los sirvientes muertos.

Vamos a ver como una idea similar se utiliza para el criptoanálisis. El método se divide en una parte para derivar la permutación de filas, y otra análoga para la permutación de columnas, luego hablaré sólo de la parte de las filas. La analogía con el problema es la siguiente:

- filas  $\equiv$  vinos
- sirvientes mínimos para resolver el problema  $\equiv$  matrices mínimas para resolver el problema
- dado vino  $v$ : bit  $j$  a 1 si el sirviente  $j$  bebe el vino  $v$ ; 0 en caso contrario  $\equiv$  dada fila  $f$ : valor  $j$  a  $w$  si la matriz  $j$  tiene la fila  $f$  con valor  $w$

Si tenemos  $m$  posibles valores para las filas, y  $M$  filas, entonces la longitud de  $M$  en base  $m$  es el techo de  $k = \log_m(M)$ , luego necesitamos mínimo  $k$  matrices para representar de manera única cada fila usando esta codificación. De esta forma, **codificamos la fila  $f$  de forma única con los valores que toma la fila  $f$  en las  $k$  matrices**.

La clave está en que, tras aplicar cifrado, las filas de cada matriz habrán cambiado de orden, pero si nos fijamos en los valores que toma la fila  $f'$  en

las  $k$  matrices cifradas podremos recuperar exactamente la fila que ocupaba  $f'$  antes del cifrado, simplemente observando qué fila codifican esos valores. De esta manera, derivamos la permutación de filas que está usando el cifrado. Aplicando el mismo procedimiento mutatis mutandis derivamos la permutación de columnas.

Como ejemplo, vamos a ver qué matrices habría que utilizar para derivar la permutación de columnas en un criptosistema con  $M = N = 4$  y un alfabeto con  $m = 2$  símbolos. Se necesitan  $k = \log_2(4) = 2$  matrices, que serán:

$$M_1 = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 \end{pmatrix}, M_2 = \begin{pmatrix} 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \end{pmatrix}.$$

Si la permutación que estamos usando es  $(3, 1, 0, 2)$  y la matriz de permutación que induce es  $P$ , entonces:

$$M_1 P^T = \begin{pmatrix} 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \end{pmatrix}, M_2 P^T = \begin{pmatrix} 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \end{pmatrix}.$$

Si leemos la primera columna de ambas matrices obtenemos  $(1, 1)_2 = 3$ ; si leemos la segunda  $(0, 1)_2 = 1$ ; la tercera  $(0, 0)_2 = 0$ ; y la cuarta  $(1, 0)_2 = 2$  (nótese como hay que invertir el orden puesto que la primera matriz es el valor de menos peso), y así recuperamos la permutación de columnas  $(3, 1, 0, 2)$ .

En cuanto a la pregunta sobre por qué se necesitan  $\log_m(MN)$  matrices, ya se ha respondido durante la exposición, pero se puede hacer una matización. Hemos visto que, para derivar la matriz de permutaciones de filas-columnas, se necesitan tantas matrices como la longitud de  $M \cdot N$  en base el tamaño del alfabeto. Si no tenemos en cuenta que el número de matrices tiene que ser entero, esto nos da  $\log_m(M) + \log_m(N) = \log_m(MN)$ , que es el valor que nos dice el enunciado. La matización que se puede hacer es que, estrictamente, se necesitan más matrices que las que dice el enunciado, puesto que el techo hay que usarlo en cada uno de los sumandos, y no en el logaritmo de la multiplicación. Por tanto, el valor exacto de matrices que hay que utilizar es

$$\lceil \log_m(M) \rceil + \lceil \log_m(N) \rceil,$$

en lugar de

$$\lceil \log_m(MN) \rceil.$$

## 8.1 Uso del programa

Dado que es el ejercicio más enjundioso y que no se daba formato para el programa, lo hemos implementado de forma mucho menos flexible que los demás. Los parámetros que se pueden variar del programa son

- $M$ , el número de filas. Este parámetro, de cambiarse, se tendrá que modificar en `opcional.h` y en `permutacion.h`, poniendo el mismo valor en ambos ficheros.
- $N$ , el número de columnas. Este parámetro, de cambiarse, se tendrá que modificar en `opcional.h` y en `permutacion.h`, poniendo el mismo valor en ambos ficheros.
- `ALPH_SIZE`, el tamaño del alfabeto. Se modifica en `opcional.h`.
- Las claves `k1` y `k2`, permutación de filas y columnas respectivamente. Se modifica en `opcional.h`.

El control de errores se ha mantenido a mínimo para no ofuscar la claridad del criptoanálisis. Se confía en la bondad del evaluador para modificar los parámetros de forma coherente: que sean números positivos, que las claves sean del tamaño adecuado...

Un ejemplo de uso para valores  $M = 8, N = 17, m = 3, K_1 = (0, 7, 5, 2, 3, 6, 1, 4), K_2 = (1, 16, 7, 12, 6, 0, 15, 4, 2, 11, 3, 5, 13, 14, 9, 8, 10)$ :

```

1 ./opcional
2 ----- Matrices fila
3 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
4 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
5 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
6 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
7 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
8 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
9 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
10 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
11
12 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
13 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
14 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
15 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
16 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
17 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
18 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
19 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
20
21 ----- Matrices fila cifradas
22 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
23 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
24 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
25 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
26 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
27 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
28 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
29 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
30
31 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
32 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
33 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
34 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
35 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
36 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
37 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
38 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
39
40 ----- Matrices columna
41
42 0 1 2 0 1 2 0 1 2 0 1 2 0 1 2 0 1

```

```

43 0 1 2 0 1 2 0 1 2 0 1 2 0 1 2 0 1
44 0 1 2 0 1 2 0 1 2 0 1 2 0 1 2 0 1
45 0 1 2 0 1 2 0 1 2 0 1 2 0 1 2 0 1
46 0 1 2 0 1 2 0 1 2 0 1 2 0 1 2 0 1
47 0 1 2 0 1 2 0 1 2 0 1 2 0 1 2 0 1
48 0 1 2 0 1 2 0 1 2 0 1 2 0 1 2 0 1
49 0 1 2 0 1 2 0 1 2 0 1 2 0 1 2 0 1
50
51 0 0 0 1 1 1 2 2 2 0 0 0 1 1 1 2 2
52 0 0 0 1 1 1 2 2 2 0 0 0 1 1 1 2 2
53 0 0 0 1 1 1 2 2 2 0 0 0 1 1 1 2 2
54 0 0 0 1 1 1 2 2 2 0 0 0 1 1 1 2 2
55 0 0 0 1 1 1 2 2 2 0 0 0 1 1 1 2 2
56 0 0 0 1 1 1 2 2 2 0 0 0 1 1 1 2 2
57 0 0 0 1 1 1 2 2 2 0 0 0 1 1 1 2 2
58 0 0 0 1 1 1 2 2 2 0 0 0 1 1 1 2 2
59
60 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1
61 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1
62 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1
63 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1
64 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1
65 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1
66 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1
67 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1
68
69 ----- Matrices columna cifradas
70
71 1 1 1 0 0 0 0 1 2 2 0 2 1 2 0 2 1
72 1 1 1 0 0 0 0 1 2 2 0 2 1 2 0 2 1
73 1 1 1 0 0 0 0 1 2 2 0 2 1 2 0 2 1
74 1 1 1 0 0 0 0 1 2 2 0 2 1 2 0 2 1
75 1 1 1 0 0 0 0 1 2 2 0 2 1 2 0 2 1
76 1 1 1 0 0 0 0 1 2 2 0 2 1 2 0 2 1
77 1 1 1 0 0 0 0 1 2 2 0 2 1 2 0 2 1
78 1 1 1 0 0 0 0 1 2 2 0 2 1 2 0 2 1
79
80 0 2 2 1 2 0 2 1 0 0 1 1 1 1 0 2 0
81 0 2 2 1 2 0 2 1 0 0 1 1 1 1 0 2 0
82 0 2 2 1 2 0 2 1 0 0 1 1 1 1 0 2 0
83 0 2 2 1 2 0 2 1 0 0 1 1 1 1 0 2 0
84 0 2 2 1 2 0 2 1 0 0 1 1 1 1 0 2 0
85 0 2 2 1 2 0 2 1 0 0 1 1 1 1 0 2 0
86 0 2 2 1 2 0 2 1 0 0 1 1 1 1 0 2 0
87 0 2 2 1 2 0 2 1 0 0 1 1 1 1 0 2 0
88
89 0 1 0 1 0 0 1 0 0 1 0 0 1 1 1 0 1
90 0 1 0 1 0 0 1 0 0 1 0 0 1 1 1 0 1
91 0 1 0 1 0 0 1 0 0 1 0 0 1 1 1 0 1
92 0 1 0 1 0 0 1 0 0 1 0 0 1 1 1 0 1
93 0 1 0 1 0 0 1 0 0 1 0 0 1 1 1 0 1
94 0 1 0 1 0 0 1 0 0 1 0 0 1 1 1 0 1
95 0 1 0 1 0 0 1 0 0 1 0 0 1 1 1 0 1
96 0 1 0 1 0 0 1 0 0 1 0 0 1 1 1 0 1
97
98 ----- Resultados
99 La permutacion de filas obtenida es:
100
101 0 7 5 2 3 6 1 4
102
103 La permutacion de columnas obtenida es:
104
105 1 16 7 12 6 0 15 4 2 11 3 5 13 14 9 8 10
106
107 ----- Fin Resultados

```

Se puede comprobar que se obtienen las permutaciones correctas.